

→ What is Parsing?

Parser: Parsing is the process of checking whether the sequence of tokens produced by lexical analysis follows the grammar of the programming language.

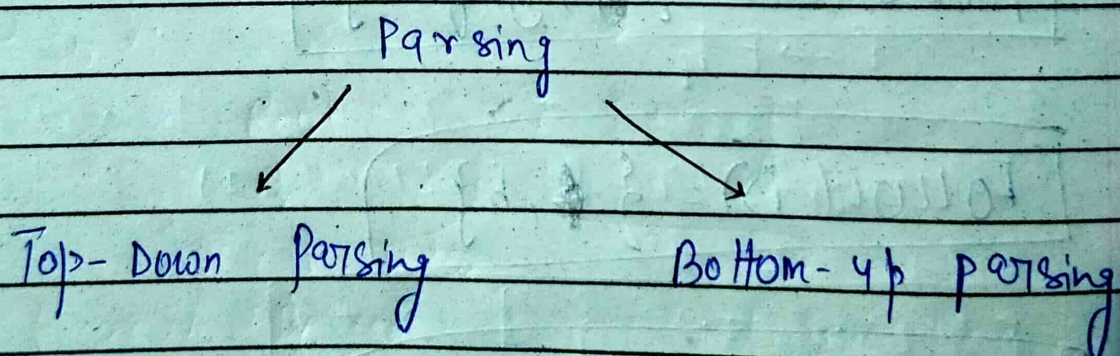
Eg:

$a+b$

A parser checks whether it follows the grammar for an expression.

The parser also builds a parse tree or another syntactic representation.

Types of parsing:



→ What is Top-Down Parsing?

Top-Down parsing is a parsing technique in which we start from the Start symbol of the grammar and try to generate the input string.

In simple words;

Start from the grammar's Start symbol → move towards the input.

eg;

$S \rightarrow aA$

$A \rightarrow b$

input:
ab

derivation:

S

↓

aA

↓

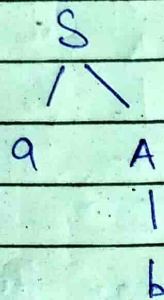
ab

We started with S and reached the input ab
∴, this is top-down parsing.



→ why it is called Top-Down?

Consider a parse Tree:



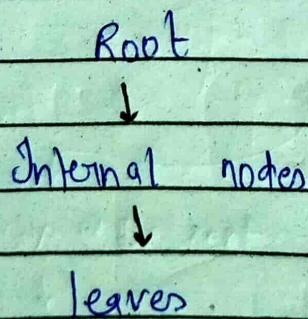
The parser start from the root :

S

move towards the leaves:

a b

So :



that is why it is called Top-Down parsing.

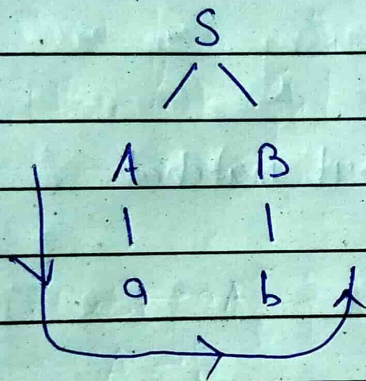
→ Top-Down parsing tries to construct a leftmost derivation.

Eg:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$



ab

→ Working:

Grammar:

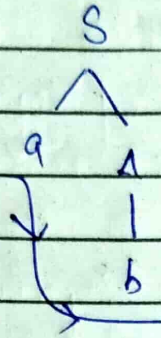
$S \rightarrow aA$

$A \rightarrow b$

input: ab



The parser starts with : S



ab

Now the complete input matched

Input = ab
Generated = ab

∴ accepted.

→ Types of TOP-Down Parsing :

Top-Down Parsing

Recursive Descent

↓
may use
backtracking

Predictive Parsing

↓
No backtracking

↓
LL(1)

→ Recursive Descent:

Recursive Descent parsing is a top-down parsing method where we create a function/procedure for each non-terminals.

Consider:

$$S \rightarrow aA$$

$$A \rightarrow b$$

we can image.

ParseS()

↓

match 'a'

↓

ParseA()

↓

~~match~~ match 'b'

So the grammar non terminals can be represent by functions.

ParseS() {

 match('a');

 ParseA();

}

ParseA() {

 match('b');

}

The exact implementation depends on the programming language, but the idea is:

One parsing procedure/function corresponds to a grammar non-terminals.

→ Backtracking:

Recursive descent parsing can use backtracking.

Consider:

$S \rightarrow ab|ac$

input: ac

The parser might first try:

$S \rightarrow ab$

but the i/p is:

ac

So

$ab \neq ac$

The parser goes back and tries:

$s \rightarrow ac$

❶ This succeeds.

This process of going back and trying another production is called backtracking.

→ Problem with backtracking:

Backtracking can be inefficient because the parser may try many possibilities.

Eg:

$A \rightarrow abc | abd | ghi | abf | \dots$

The parser may have to try several alternatives.

∴ we prefer **predictive** parsing, where the parser can select the correct production directly.

→ Predictive parsing :
Predictive parsing is a type of top-down parsing that tries to select the correct production without backtracking.

It looks at the upcoming i/p symbol and predicts which production should be used.

It is also called Non-Recursive Predictive Parsing.

